

最佳化決策演算法

作者：羅浚嘉

單位：國立澎湖科技大學 資工系2甲

學號:1113405040

開發環境：Java

Email:d13405040@ems.npu.edu.tw

摘要 (Abstract)

本實驗旨在使用 Java 程式語言實現兩種經典演算法：最短路徑演算法（包含 Dijkstra 與 Bellman-Ford 演算法）與動態規劃（0/1 背包問題）。透過對不同圖形結構（正權重、負權重邊）以及決策矩陣的求解，成功計算出最短路徑之標籤值與背包最大價值（4）；並透過優先佇列與二維矩陣的最佳化，實現高效率的數據運算。本報告詳細敘述其邏輯實現、時間複雜度分析及執行結果。

關鍵字：最短路徑、Dijkstra 演算法、Bellman-Ford 演算法、動態規劃、0/1 背包問題、時間複雜度

程式碼在：

<https://github.com/d13405040-boop/1113405040>

在演算法領域中，如何有效分配資源、尋找網路架構中的最短路徑，以及在有限限制下進行資源最佳化配置，皆是核心課題。

本實驗探討三大核心問題：首先是 Dijkstra 演算法，其在所有邊均為正權重的情況下，能快速找出單源最短路徑；然而，當網路中存在負權重的邊時，Dijkstra 演算法將失效，此時必須引入 Bellman-Ford 演算法進行全局無差別鬆弛以確保正確性。最後，針對資源分配的經典 0/1 背包問題，本實驗捨棄了暴力搜尋法，改採動態規劃 (Dynamic Programming) 策略，利用子問題的最佳解逐步建構出全局最佳解。本實驗透過 Java 實作，驗證上述理論演算法在實際測資下的正確性與效率。

I. 前言 (INTRODUCTION)

II. 方法與實驗結果 (METHOD AND EXPERIMENTAL RESULTS)

1. Dijkstra 最短路徑演算法 (Dijkstra's Algorithm)

方法描述：首先建立圖形結構，利用鄰接清單儲存節點與正權重邊。定義 Node 類別並實作 Comparable 介面。利用優先佇列 (PriorityQueue) 每次提取出當前距離起點最短的未鎖定節點，並對其鄰接的邊進行鬆弛嘗試，直到所有節點皆被鎖定納入確定集合中。

實驗測資與推進過程：使用 6 個節點 ($V=6$)、16 條正權重雙向邊 ($E=16$) 之圖形。起點設為節點 1。系統依序鎖定節點 1、2、5、3、4、6，最終求得起點 1 到各節點的最短路徑長度分別為：節點 1 距離為 0、節點 2 距離為 4、節點 3 距離為 6、節點 4 距離為 9、節點 5 距離為 5、節點 6 距離為 11。

時間複雜度分析： $O((V + E) \log V)$ 。因為透過 PriorityQueue 每次取出當前最短距離點需 $O(\log V)$ ，並對每條邊進行鬆弛嘗試。

空間複雜度分析： $O(V + E)$ 。用於儲存鄰接清單與距離陣列。

2. Bellman-Ford 最短路徑演算法 (Bellman-Ford Algorithm)

方法描述：為解決含負權重邊的最短路徑問題，此演算法不採用貪婪策略，而是進行最多 $V-1$ 輪的全局無差別鬆弛。每一輪皆對圖中所有的邊進行一次檢查，若發現更短路徑則更新距離陣列。

實驗測資與推進過程：使用 4 個節點 ($V=4$)、6 條邊 ($E=6$ ，包含負權重邊) 之圖形。起點設為節點 1。本實驗歷經 3 輪邊的無差別疊加鬆弛，最終陣列成功收斂。起點 1 到各節點的最短距離分別為：節點 1 最短距離為 0、節點 2 最短距離為 -6、節點 3 最短距離為 -4、節點 4 最短距離為 5。

時間複雜度分析： $O(V \times E)$ 。外層迴圈固定掃描 $V-1$ 輪，內層每輪皆掃描全部的 E 條邊。正因為這種全局視野，才能正確處理負值路徑。

空間複雜度分析： $O(V + E)$ 。用於儲存邊的集合與距離陣列。

3. 0/1 背包問題動態規劃 (0/1 Knapsack Problem - DP)

方法描述：建立大小為 $(N+1) \times (W+1)$ 的二維狀態矩陣表格 $dp[i][cap]$ 。外層迴圈疊代物品，內

層迴圈疊代背包限重。核心決策邏輯為：當前背包容量足夠時，決策是否放入該物品，即取「不放該物品：dp[i-1][cap]」與「放入該物品並加上剩餘限重最佳解：dp[i-1][cap-w] + v」兩者的最大值。

實驗測資與數據輸出：物品數 N=3、背包限重 W=4。

物品 \ 背包限重 0 1 2 3 4 初始狀態
(0) 0 0 0 0 0 物品 1 (重 2, 值 3) 0 0 3 3
3 物品 2 (重 3, 值 4) 0 0 3 4 4 物品 3
(重 1, 值 2) 0 2 3 5 6

最佳化決策產出：背包能裝載的最大價值為 6。

時間複雜度分析： $O(N \times W)$ 。透過雙層嵌套迴圈，精準填滿二維決策矩陣，每格計算時間為 $O(1)$ 。

空間複雜度分析： $O(N \times W)$ 。用於配置儲存子問題最佳解的二維陣列表格空間

```
-----【結果一】Dijkstra 演算法：節點確定推進過程-----
[確定] 確定集合加入節點 [1] -> 目前最短距離為：0
[確定] 確定集合加入節點 [2] -> 目前最短距離為：4
[確定] 確定集合加入節點 [5] -> 目前最短距離為：5
[確定] 確定集合加入節點 [3] -> 目前最短距離為：6
[確定] 確定集合加入節點 [4] -> 目前最短距離為：9
[確定] 確定集合加入節點 [6] -> 目前最短距離為：11

-----【結果二】Dijkstra 演算法：最終最短距離陣列（權重值）-----
起點 1 到節點 1 的最短距離長度 = 0
起點 1 到節點 2 的最短距離長度 = 4
起點 1 到節點 3 的最短距離長度 = 6
起點 1 到節點 4 的最短距離長度 = 9
起點 1 到節點 5 的最短距離長度 = 5
起點 1 到節點 6 的最短距離長度 = 11

[?? 複雜度分析 - Dijkstra]
? 時間複雜度:  $O((V + E) \log V)$  -> 本範例頂點 V=6, 邊 E=16
- 原因: 透過 PriorityQueue 每次取出當前最距離點需  $O(\log V)$ , 並對每條邊進行鬆弛嘗試。
? 空間複雜度:  $O(V + E)$  -> 用於儲存鄰接清單與距離陣列。

-----【結果三】Bellman-Ford 演算法：逐輪全局無差別掃描（對應截圖一）-----
[第 1 輪: 邊的無差別疊加]
發現更短距離! 節點 1 -> 2: 原距離 = 改為 6
發現更短距離! 節點 1 -> 3: 原距離 = 改為 5
發現更短距離! 節點 1 -> 4: 原距離 = 改為 5
發現更短距離! 節點 2 -> 3: 原距離 5 改為 4
發現更短距離! 節點 3 -> 2: 原距離 6 改為 2
發現更短距離! 節點 4 -> 3: 原距離 4 改為 3
[第 2 輪: 邊的無差別疊加]
發現更短距離! 節點 2 -> 3: 原距離 3 改為 0
發現更短距離! 節點 3 -> 2: 原距離 2 改為 -2
[第 3 輪: 邊的無差別疊加]
發現更短距離! 節點 2 -> 3: 原距離 0 改為 -4
發現更短距離! 節點 3 -> 2: 原距離 -2 改為 -6

--- Bellman-Ford 最終收斂陣列 ---
起點 1 到節點 1 的最短距離 = 0
起點 1 到節點 2 的最短距離 = -6
起點 1 到節點 3 的最短距離 = -4
起點 1 到節點 4 的最短距離 = 5

[?? 複雜度分析 - Bellman-Ford]
? 時間複雜度:  $O(V \times E)$  -> 本範例頂點 V=4, 邊 E=6
- 原因: 外層迴圈固定掃描 V-1 輪, 內層迴圈每輪皆無差別掃描全部的 E 條邊。正因為這種全局視野, 才能正確處理【負值】鬆弛。
? 空間複雜度:  $O(V + E)$  -> 用於儲存邊的集合與距離陣列。

-----【結果四】0/1 背包問題：動態規劃決策矩陣（對應截圖二）-----
[DP 二維矩陣填表狀態輸出]
物品 \ 背包限重 0 1 2 3 4
-----
初始狀態 (0) 0 0 0 0 0
物品1 (重2,值3) 0 0 3 3 3
物品2 (重3,值4) 0 0 3 4 4
物品3 (重1,值2) 0 2 3 5 6

? 最佳化決策產出: 背包能裝載的最大價值為 = 6

[?? 複雜度分析 - 0/1 背包問題動態規劃]
? 時間複雜度:  $O(N \times W)$  -> 本範例物品數 N=3, 背包限重 W=4
- 原因: 透過雙層嵌套迴圈, 精準填滿一個大小為  $(N+1) \times (W+1)$  的二維決策矩陣, 每格計算時間為  $O(1)$ 。
? 空間複雜度:  $O(N \times W)$  -> 用於配置儲存子問題最佳解的二維陣列表格空間。
```

III. 結論 (CONCLUSION)

本實驗成功利用 Java 語言實作了單源最短路徑演算法與動態規劃決策。實驗結果證明：

在無負權重邊的網路中，Dijkstra 演算法透過優先佇列能將時間複雜度有效控制，並精確鎖定各節點的最短路徑。

面對圖形中含有負權重邊時，Bellman-Ford 演算法雖時間複雜度較高（ $O(V \times E)$ ），但其全局鬆弛機制

能確保在負值環境下輸出正確的最短路徑陣列。本實驗測資經過 3 輪鬆弛後成功收斂，精準計算出包含負值在內的最短距離。

在 0/1 背包問題中，利用動態規劃（DP）的二維矩陣填表法，能避免暴力法帶來的指數級時間複雜度，將其優化至 $O(N \times W)$ 。在本實驗設定之物品條件與限重 4 的限制下，成功產出最大總價值為 6 的最佳化決策。